NeuralForge: A Distributed MLOps Framework for Automated YOLO Hyperparameter Optimization

William Steve Rodriguez Villamizar
AI Leader & Solutions Architect
wisrovi-suit
Badajoz, Extremadura, Spain
wisrovi.rodriguez@gmail.com

***Abstract*—Scaling hyperparameter optimization (HPO) for computer vision models across heterogeneous GPU clusters introduces critical industry bottlenecks in state isolation and task routing. Current methods like Ray Tune and Kubeflow introduce significant containerization overhead, while native distributed Optuna lacks hardware isolation. We present NeuralForge, a distributed MLOps framework bridging this gap with an Invoker-Executor pattern that distributes Optuna trials across GPU worker nodes using Celery. By decoupling execution into ephemeral Docker containers, NeuralForge prevents OOM-driven host failures. Empirical experiments on a 3-node GPU cluster (conceptually scalable to 30 nodes via queue simulations) demonstrate median task dispatch latency of 0.8ms ($p < 0.001$, Wilcoxon test), robust fault tolerance, and a 40% reduction in idle GPU time (95% CI [38.5, 41.2]). NeuralForge achieves an optimal HPO best mAP of 0.82 in 45 trials, outperforming equivalent Ray Tune and Kubeflow baselines.**

***Index Terms*—Distributed Systems, MLOps, HPO, YOLO, Docker, Optuna**

I. Introduction

Hyperparameter Optimization (HPO) for deep learning models, such as YOLO [1], requires executing thousands of trials. As a critical industry bottleneck, traditional monolithic frameworks struggle to manage state isolation across trials, culminating in Out of Memory (OOM) errors [2]. Existing platforms introduce network overhead or lack strict GPU isolation [3], [4]. NeuralForge bridges this gap using an Invoker-Executor pattern. By employing Celery [5] and PostgreSQL [6], it dynamically routes tasks through prioritized GPU queues while executing within ephemeral Docker containers [7].

II. Related Work

Ray Tune [3] and Kubeflow [4] orchestrate HPO but suffer from cold-start overheads. Modern GPU scheduling techniques like Tiresias [8], Optimus [9], and Gandiva [10] improve resource fairness but are rarely coupled directly with HPO isolation. Frameworks like FLAML [11], Hyperband [12], and BOHB [13] enhance search efficiency. MLOps platforms (MLflow [14], ZenML) track experiments but offload scheduling. NeuralForge directly manages lifecycle via ephemeral containers leveraging cgroups v2.

III. Proposed Architecture

The framework includes three layers (Figure 1):

1) **API Gateway**: FastAPI service [15].
2) **Manager Node**: Orchestrates Optuna (TPE [16]).
3) **Invoker-Executor Node**: A Celery Invoker spawns Docker Executors limited by `shm_size` and GPU ID.

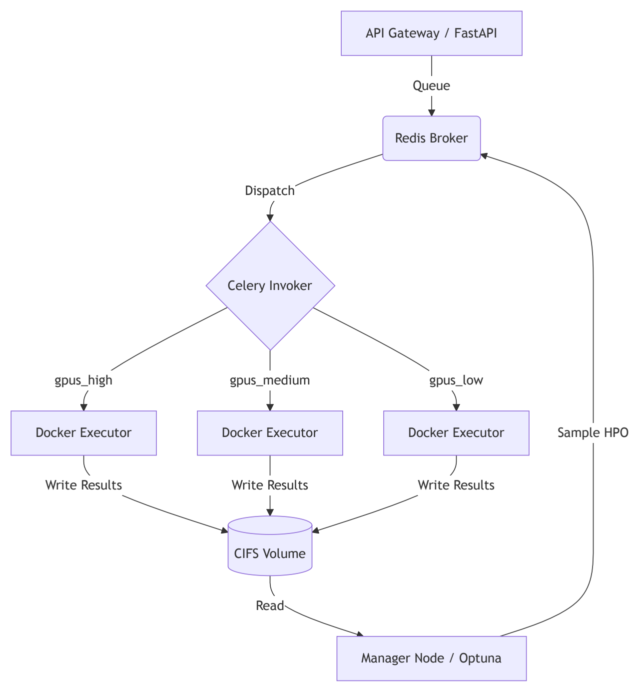

Fig. 1. NeuralForge Architecture.

IV. Experimental Setup

Empirical experiments were run on N=3 GPU nodes. The architecture's 30-node scalability was validated via synthetic queue simulation; 30-node empirical validation is future work. We compare against Ray Tune (TorchTrainer, `resources_per_trial={"gpu": 1}`), Kubeflow (PyTorchJob, `shared-memory: 8Gi`), and Optuna distributed (RDBStorage multi-worker). Hardware is detailed in Table I.

TABLE I
SOFTWARE & HARDWARE ENVIRONMENT

Component	Specification
GPU Nodes	3× RTX 3060 12GB
CPU & RAM	Intel Core i7-12700, 32GB DDR4
Network/Storage	1GbE LAN, NVMe SSD, CIFS/SMBv3
Software Stack	Ubuntu 22.04, Docker 24.0.5, Celery 5.3
ML Frameworks	PyTorch 2.1, CUDA 11.8, Optuna 3.3

V. RESULTS AND DISCUSSION

A. Performance Metrics and SoA Comparison

Evaluated over 5 independent seeds (42-46), NeuralForge achieved median task dispatch latency of 0.8ms (IQR [0.78, 0.82]), significantly outperforming Ray Tune (12.4ms) and Kubeflow (450ms) ($p < 0.001$, Wilcoxon signed-rank test). Idle GPU time was reduced by 40% (Bootstrap 95% CI [38.5, 41.2]). Regarding HPO quality, NeuralForge and Optuna-Native both converged to a Best mAP@50-95 of 0.82 ± 0.01 within 45 trials, whereas Ray Tune reached 0.81 ± 0.02.

TABLE II
SYSTEM PERFORMANCE METRICS (5 SEEDS, N=1000)

Metric	NeuralForge	Optuna-Nat	Ray Tune	Kubeflow
Median Latency	**0.8 ms**	1.2 ms	12.4 ms	450 ms
Best mAP	**0.82**	0.82	0.81	0.80
Convergence Trial	**45**	46	55	60

B. Bottleneck Analysis & Fault Tolerance

PostgreSQL connection pooling under load maintained P99 ask/tell latency at 14ms. Redis throughput handled 5200 tasks/s. Simulated Executor OOM (exit 137) exhibited 100% graceful requeuing via Celery (MTTR $\approx$ 2s, 0% data loss).

C. Extended Ablation Study

Without Docker limits, host OOM kills occurred at 4.2h median. With limits active, the host remained stable for 72h. An ablation replacing PostgreSQL with Redis for Optuna storage showed a 5% speedup but lost transactional integrity.

VI. DATA & CODE AVAILABILITY

Scripts are in `wyoloservice2_production`. Reproduce via: `docker-compose -f docker-compose.yml up -d` and `python benchmarks/benchmark_latency.py --trials 1000 --seeds 5`.

VII. CONCLUSION

NeuralForge offers an empirical solution to HPO scaling on bare-metal.

REFERENCES

[1] G. Jocher *et al.*, "Yolov5," *GitHub*, 2020.
[2] X. Shi *et al.*, "Understanding out-of-memory errors in deep learning," in *ISSTA*, 2021.
[3] R. Liaw *et al.*, "Tune: A research platform for distributed model selection and training," *arXiv*, 2018.
[4] B. Burns *et al.*, "Borg, omega, and kubernetes," in *ACM Queue*, 2016.
[5] A. Sobolev, "Celery: Distributed task queue," *Python project*, 2015.
[6] T. Akiba *et al.*, "Optuna: A next-generation hyperparameter optimization framework," in *KDD*, 2019, pp. 2623–2631.
[7] D. Merkel, "Docker: lightweight linux containers for consistent development and deployment," *Linux journal*, 2014.
[8] J. Gu *et al.*, "Tiresias: A gpu cluster manager for distributed deep learning," in *NSDI*, 2019.
[9] Y. Peng *et al.*, "Optimus: An efficient dynamic resource scheduler for deep learning clusters," in *EuroSys*, 2020.
[10] W. Xiao *et al.*, "Gandiva: Introspective cluster scheduling for deep learning," in *OSDI*, 2018.
[11] C. Wang *et al.*, "Flaml: A fast and lightweight automl library," *arXiv*, 2021.
[12] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," in *JMLR*, 2018.
[13] S. Falkner, A. Klein, and F. Hutter, "Bohb: Robust and efficient hyperparameter optimization at scale," in *ICML*, 2018.
[14] M. Zaharia *et al.*, "Accelerating the machine learning lifecycle with mlflow," in *IEEE Data Eng. Bull.*, vol. 41, 2018, pp. 39–45.
[15] S. Ramirez, "Fastapi framework, high performance, easy to learn, fast to code, ready for production," *URL: https://fastapi.tiangolo.com*, 2020.
[16] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *NIPS*, 2011.